

Bienvenue au cours “C,C#,.NET Intermediaire”

Nom de l’enseignant : Fabrice Meynckens

Promo : 2025-2026

Classe : B3CDEV



Objectifs de la session

- Linq : group by
- Jointures en Linq
- Exercice Linq complémentaire (jointures)
- Compréhension de la programmation asynchrone en C#
- Mécanismes async/await
- Fire and Forget
- Gestion des tâches (task)

Linq : group by

L'opérateur **GroupBy()** en LINQ est utilisé pour diviser les éléments d'une collection en groupes basés sur une clé commune. C'est l'équivalent logique de la clause **GROUP BY** en SQL.

Voici des exemples concrets pour illustrer différentes utilisations de **GroupBy()**.

Regroupement Simple par Propriété

Cet exemple montre comment regrouper une liste d'étudiants par leur **Département**.

Le contexte

```
public class Etudiant
{
    public string Nom { get; set; }
    public string Departement { get; set; }
    public int Age { get; set; }
}

List<Etudiant> etudiants = new List<Etudiant>
{
    new Etudiant { Nom = "Alice", Departement = "Informatique", Age = 20 },
    new Etudiant { Nom = "Bob", Departement = "Physique", Age = 22 },
    new Etudiant { Nom = "Charlie", Departement = "Informatique", Age = 21 },
    new Etudiant { Nom = "Diana", Departement = "Physique", Age = 20 },
    new Etudiant { Nom = "Eve", Departement = "Maths", Age = 23 }
};
```

La Requête **GroupBy** (Syntaxe de Méthode)

```
var etudiantsParDepartement = etudiants.GroupBy(e => e.Departement);

Console.WriteLine("--- Étudiants regroupés par Département ---");
foreach (var groupe in etudiantsParDepartement)
{
    // 'groupe.Key' est la valeur de la propriété utilisée pour le regroupement
    Console.WriteLine($"\\nDépartement: {groupe.Key} ({groupe.Count()} étudiants)");

    // 'groupe' est lui-même une collection (IEnumerable<Etudiant>) des éléments
    foreach (var etudiant in groupe)
    {
        Console.WriteLine($"  - {etudiant.Nom} ({etudiant.Age} ans)");
    }
}
```

Linq : group by avec agrégation

Le `GroupBy` est très souvent suivi d'une projection (`Select`) pour calculer une statistique par groupe. Cet exemple calcule l'**âge moyen** par département.

```
var moyenneAgeParDepartement = etudiants
    .GroupBy(e => e.Dpartement)
    .Select(groupe => new
    {
        Dpartement = groupe.Key,
        // Utilisation de l'opérateur Average sur la collection du groupe
        AgeMoyen = groupe.Average(e => e.Age)
    });

Console.WriteLine("\n--- Âge Moyen par Département ---");
foreach (var item in moyenneAgeParDepartement)
{
    Console.WriteLine($"- {item.Dpartement}: {item.AgeMoyen:F1} ans");
}
```

Exercice Linq : group by

Exercice : Analyse de Stock de Fruits 🍎 🍌

Vous travaillez pour un marché qui doit rapidement analyser son stock de fruits. Chaque fruit a un nom et une couleur.

1. Le Contexte

Voici la liste des fruits en stock :

```
public class Fruit
{
    public string Nom { get; set; }
    public string Couleur { get; set; }
}

List<Fruit> stockFruits = new List<Fruit>
{
    new Fruit { Nom = "Pomme", Couleur = "Rouge" },
    new Fruit { Nom = "Banane", Couleur = "Jaune" },
    new Fruit { Nom = "Fraise", Couleur = "Rouge" },
    new Fruit { Nom = "Citron", Couleur = "Jaune" },
    new Fruit { Nom = "Pomme", Couleur = "Vert" },
    new Fruit { Nom = "Cerise", Couleur = "Rouge" }
};
```

L'Objectif LINQ

Utilisez l'opérateur **GroupBy()** pour :

1. **Regrouper** la liste `stockFruits` par la propriété **Couleur**.
2. **Afficher** chaque couleur trouvée.
3. Pour chaque couleur, **compter** le nombre de fruits associés et lister leurs **noms**.

Jointures en Linq

Faire des requêtes LINQ avec des **jointures** est essentiel pour combiner des données provenant de deux sources ou collections distinctes, tout comme on le ferait avec une clause **JOIN** en SQL.

En LINQ, vous avez deux syntaxes principales pour effectuer des jointures : la **Syntaxe de Requête** (plus proche du SQL) et la **Syntaxe de Méthode** (plus courante en C# moderne).

Principe de la Jointure (**Join**)

La méthode standard de jointure en LINQ est l'**Inner Join** (jointure interne). Elle retourne uniquement les éléments qui ont une correspondance dans **les deux** collections, basée sur une clé commune.

Pour réaliser une jointure, vous devez spécifier quatre éléments clés :

1. La **collection externe** (celle qui appelle la méthode ou qui est dans le **from**).
2. La **collection interne** (celle qui est jointe).
3. La **clé externe** (la propriété utilisée pour la correspondance dans la première collection).
4. La **clé interne** (la propriété utilisée pour la correspondance dans la seconde collection).
5. Le **résultat** (la projection de ce que vous voulez retourner une fois la correspondance établie).

Jointures Linq : syntaxe de méthode

Syntaxe de Méthode (**.Join()**) : La Plus Courante

La syntaxe de méthode utilise la méthode d'extension **.Join()** et des expressions lambda pour définir les clés et la projection.

Imaginez que vous joignez une liste d'**Employés** avec une liste de **Départements** via l'ID du département.

Explication de la Projection

La projection finale (**employe, departement**) => **new { ... }** est une expression lambda qui prend l'objet de la collection externe (**employe**) et l'objet de la collection interne (**departement**) qui correspondent, et vous permet de construire un nouvel objet anonyme contenant les données combinées.

```
// Employés (Collection Externe)
List<Employe> Employes = new List<Employe> {
    new Employe { Nom = "Alice", DepId = 1 },
    new Employe { Nom = "Bob", DepId = 2 }
};

// Départements (Collection Interne)
List<Departement> Departements = new List<Departement> {
    new Departement { Id = 1, NomDep = "Ventes" },
    new Departement { Id = 2, NomDep = "Marketing" }
};

var resultatJointure = Employes.Join(
    // 1. Collection interne :
    Departements,

    // 2. Clé externe (Employe) :
    employe => employe.DepId,

    // 3. Clé interne (Departement) :
    departement => departement.Id,

    // 4. Projection (Ce que l'on retourne pour chaque paire correspondante) :
    (employe, departement) => new
    {
        NomEmploye = employe.Nom,
        NomDepartement = departement.NomDep
    }
);

// Résultat : { NomEmploye = "Alice", NomDepartement = "Ventes" },
//           { NomEmploye = "Bob", NomDepartement = "Marketing" }
```

Syntaxe de Requête (SQL-like) : La Plus Lisible

La syntaxe de requête utilise les mots-clés `join...in...on...equals...select`, ce qui la rend très familière aux développeurs SQL.

```
var resultatJointure = from employe in Employes // Collection Externe
                        join departement in Departements // Collection Interne
                        on employe.DepId equals departement.Id // Condition de joint
                        select new // Projection du résultat
                        {
                            NomEmploye = employe.Nom,
                            NomDepartement = departement.NomDep
                        };

```

```
// Le résultat est identique à la syntaxe de méthode.
```


Exercice Linq avec jointure

Vous travaillez sur un système de gestion interne qui suit les employés et leurs projets. Vous devez générer un rapport combinant ces deux sources de données.

Les Questions (Objectifs LINQ)

1. **Jointure et Projection (Join & Select) :**
 - Joindre les deux collections en utilisant la clé commune (**Employee.Id** et **Projet.ResponsableId**).
 - Projeter le résultat pour afficher le **Nom du Projet**, le **Nom du Responsable** et le **Département** du responsable.
2. **Filtrage sur la Jointure (Where) :**
 - Partir du résultat de la jointure (Question 1) et filtrer pour n'inclure que les projets du département "**Développement**".
3. **Groupement et Agrégation (GroupBy & Sum) :**
 - Regrouper la collection **Projets** par la propriété **ResponsableId**.
 - Calculer le **budget total** géré par chaque responsable.
 - Afficher le Nom du Responsable et son budget total (vous devrez effectuer une seconde jointure ou utiliser un dictionnaire pour afficher le nom plutôt que l'ID).

```
public class Employee
{
    public required int Id { get; set; }
    public required string Nom { get; set; }
    public required string Departement { get; set; }
}

public class Projet
{
    public required int Id { get; set; }
    public required string NomProjet { get; set; }
    public int ResponsableId { get; set; } // Clé étrangère vers Employee.Id
    public int Budget { get; set; }
}

public class LinqAvance
{
    public static void Main()
    {
        // Données (répétées ici pour la complétude du code exécutable)
        List<Employee> Employees = new List<Employee>
        {
            new Employee { Id = 101, Nom = "Alice Martin", Departement = "Développement" },
            new Employee { Id = 102, Nom = "Bob Dupont", Departement = "Ventes" },
            new Employee { Id = 103, Nom = "Charlie Noel", Departement = "Développement" },
            new Employee { Id = 104, Nom = "Diana Pomp", Departement = "Marketing" }
        };

        List<Projet> Projets = new List<Projet>
        {
            new Projet { Id = 1, NomProjet = "Migration Cloud", ResponsableId = 101, Budget = 50000 },
            new Projet { Id = 2, NomProjet = "Lancement Produit A", ResponsableId = 104, Budget = 120000 },
            new Projet { Id = 3, NomProjet = "Refonte Site Web", ResponsableId = 101, Budget = 80000 },
            new Projet { Id = 4, NomProjet = "Campagne Publicitaire", ResponsableId = 104, Budget = 30000 },
            new Projet { Id = 5, NomProjet = "Optimisation DB", ResponsableId = 103, Budget = 60000 },
            new Projet { Id = 6, NomProjet = "Analyse Marché", ResponsableId = 999, Budget = 40000 }
        };
    }
}
```

Programmation asynchrone en C#

La programmation **asynchrone** en C# est un modèle de conception qui permet à votre application de gérer les tâches longues sans bloquer le fil d'exécution (thread) principal. Cela est crucial pour maintenir la **réactivité** des applications, notamment celles dotées d'une interface utilisateur (UI) ou celles qui effectuent des opérations d'entrée/sortie (I/O).

Les mots-clés clés de ce modèle, introduits avec C# 5, sont **async** et **await**.

Pourquoi la Programmation Asynchrone ? 🤔

En programmation synchrone classique, une méthode s'exécute de manière séquentielle. Si une ligne de code effectue une opération longue (comme télécharger un fichier, accéder à une base de données, ou attendre une réponse réseau), le thread reste **bloqué** et ne peut rien faire d'autre jusqu'à ce que cette opération soit terminée.

- **Problème en UI** : L'interface utilisateur devient figée et l'application semble gelée ("Not Responding").
- **Problème en serveur (ASP.NET)** : Le thread du serveur est bloqué inutilement à attendre une réponse I/O, gaspillant des ressources qui pourraient servir à traiter d'autres requêtes.

La programmation asynchrone permet à la tâche longue de s'exécuter en arrière-plan pendant que le thread original est **libéré** pour effectuer d'autres tâches utiles.

Mécanismes async et await

Le Mot-clé `async` (La Promesse)

Le mot-clé `async` est utilisé pour **marquer une méthode** comme asynchrone.

- Il ne rend pas la méthode asynchrone en soi, mais indique au compilateur que cette méthode contiendra une ou plusieurs expressions `await`.
- Une méthode `async` doit retourner `Task`, `Task<T>`, ou `void` (ce dernier étant généralement réservé aux gestionnaires d'événements).

Le Mot-clé `await` (La Pause et la Reprise)

Le mot-clé `await` est le cœur du mécanisme. Il est placé devant une tâche asynchrone qui retourne un objet `Task`.

- Lorsque le code atteint l'instruction `await`, il met la méthode **en pause**.
- Le contrôle est **immédiatement rendu** à l'appelant de la méthode. Le thread original est libéré.
- L'exécution de la méthode reprend **à partir de ce point** lorsque la tâche "awaitée" est terminée.

Déroutement Séquentiel Illusionniste

L'avantage majeur est que, pour le développeur, le code asynchrone s'écrit de manière simple et séquentielle, comme le code synchrone, ce qui maintient la lisibilité.

```
public static async Task Main(string[] args)
{
    Console.WriteLine("Démarrage du programme principal.");

    // 1. Appel de la méthode asynchrone en utilisant 'await'
    string resultat = await TelechargerDonneesAsync("http://example.com");

    Console.WriteLine($"{\nRéception du contenu (longueur:
{resultat.Length}).");
    Console.WriteLine("Fin du programme principal.");
}

// Votre méthode asynchrone (pour référence)
public static async Task<string> TelechargerDonneesAsync(string url)
{
    Console.WriteLine($"{ -> Tâche asynchrone : Début du téléchargement de
{url}...");
    // Simulation d'une tâche longue I/O (réseau)
    await Task.Delay(2000);
    Console.WriteLine($"{ <- Tâche asynchrone : Téléchargement terminé.");

    return "Contenu simulé très long.";
}
```

Fire and Forget

Dans l'exemple précédent de la méthode `async Task Main()`, l'utilisation du mot-clé `await` force le programme à **suspendre** son exécution à ce point jusqu'à ce que la tâche asynchrone soit terminée. Bien que le thread soit libéré, l'exécution séquentielle du code C# attend la complétion.

Pour obtenir une véritable exécution "**feu et oubli**" (*fire-and-forget*), où le `Main` appelle la tâche et continue immédiatement sans attendre la réponse, vous devez **omettre le mot-clé `await`**.

Voici comment modifier l'approche pour obtenir une exécution non bloquante et simultanée (ou concurrente, dans le contexte d'une seule application).

Exécution Non Bloquante ("Feu et Oubli") 🔥

Pour que le programme principal appelle `TelechargerDonneesAsync` et continue immédiatement, vous devez appeler la méthode asynchrone, qui retourne un objet `Task`, et **ne pas utiliser `await`** sur cette `Task`.

Le Problème de l'Application Console

Dans une application console, si le thread principal (le `Main`) se termine, toute tâche asynchrone qui n'est pas "attendue" (avec `await`) ou bloquée (avec `.Wait()`) est généralement **arrêtée prématurément** par le système d'exploitation.

Pour contourner cela, nous allons stocker la référence à la tâche, la lancer, et seulement à la **fin** du `Main`, nous l'attendrons pour nous assurer qu'elle a le temps de se terminer.

1. Démarrage du programme principal (Main).
-> Tâche asynchrone : Début du téléchargement...
2. Le Main continue son exécution IMMÉDIATEMENT.
-> Tâche principale : Effectue d'autres travaux...
-> Tâche principale : Progression 1/3
-> Tâche principale : Progression 2/3
-> Tâche principale : Progression 3/3
3. Le Main a terminé ses autres travaux. Attente du téléchargement...
-< Tâche asynchrone : Données reçues et traitées.
4. Les données ont été traitées. Fin du programme.

```
public static async Task Main()
{
    Console.WriteLine("1. Démarrage du programme principal (Main).");

    // --- Exécution NON BLOQUANTE ("Fire-and-Forget") ---
    // On appelle la méthode asynchrone et on stocke l'objet Task,
    // mais on n'utilise PAS 'await'.
    Task telechargementTask = TelechargerDonneesAsync(2000);

    Console.WriteLine("\n2. Le Main continue son exécution IMMÉDIATEMENT.");
    Console.WriteLine("    -> Tâche principale : Effectue d'autres travaux...");

    // Simulation d'un travail synchrone pendant que la tâche asynchrone tourne en
    // arrière-plan
    for (int i = 0; i < 3; i++)
    {
        await Task.Delay(300); // Petite pause pour simuler un travail
        Console.WriteLine($"    -> Tâche principale : Progression {i + 1}/3");
    }

    Console.WriteLine("\n3. Le Main a terminé ses autres travaux. Attente du
    téléchargement...");

    // À la fin, on utilise AWAIT pour garantir que la tâche se termine
    // avant que l'application console ne se coupe.
    await telechargementTask; // On attend la fin de la tâche asynchrone
    Console.WriteLine("4. Les données ont été traitées. Fin du programme.");
}

public static async Task TelechargerDonneesAsync(int delay)
{
    Console.WriteLine($"    -> Tâche asynchrone : Début du téléchargement...");

    // Tâche asynchrone I/O : Simule un délai de 2 secondes
    await Task.Delay(2000);

    Console.WriteLine("    <- Tâche asynchrone : Données reçues et traitées.");
}
```

Exercice 1 : méthodes asynchrones

Préparation d'une Tasse de Thé

Le programme simule la préparation d'une tasse de thé, qui nécessite une attente (le temps pour l'eau de bouillir). Pendant ce temps, l'utilisateur effectue une tâche connexe (préparer une tasse, y mettre du thé), puis attend que le thé infuse pour pouvoir commencer la dégustation.

1. Objectifs

1. Créer une méthode asynchrone `FaireBouillirEauAsync()` qui simule un délai de **4 secondes**.
2. Dans la méthode principale (`Main`), **lancer** la tâche `FaireBouillirEauAsync()` sans utiliser `await` immédiatement (modèle "Fire-and-Forget").
3. Pendant que l'eau bout en arrière-plan, simuler une tâche synchrone dans le `Main` (`PreparerPetitDejeuner()`) qui dure **3 secondes**.
4. À la fin du `Main`, utiliser le mot-clé `await` sur la tâche de l'eau pour garantir que l'application ne se termine pas avant que l'eau ne soit prête.

Asynchrone avancé

Pour maîtriser la programmation asynchrone en C# au-delà des bases (`async/await`), il est crucial d'approfondir les notions suivantes, qui touchent à la gestion des performances, au contrôle du flux et à la gestion des erreurs.

La Gestion des Tâches (`Task`)

A. Le Type `ValueTask<T>` (Performance)

Alors que la plupart des méthodes asynchrones retournent `Task<T>`, le type `ValueTask<T>` est une structure légère conçue pour les scénarios où la tâche est susceptible de se terminer **rapidement** et de manière **synchrone** (sans attente).

- **Utilité** : Il permet d'éviter l'allocation d'un objet `Task` sur le tas (heap) à chaque appel de méthode, réduisant ainsi la pression sur le *Garbage Collector* et améliorant les performances dans les bibliothèques et les chemins de code chauds (très sollicités).

B. Combinaison et Attente Multiple (`Task.WhenAll` et `Task.WhenAny`)

Ces méthodes permettent de gérer le flux de plusieurs tâches lancées simultanément :

- `Task.WhenAll()` : Prend une collection de `Task` et retourne une seule `Task` qui se termine **uniquement lorsque TOUTES les tâches d'origine sont terminées**. C'est idéal pour l'exécution parallèle de plusieurs opérations indépendantes.
- `Task.WhenAny()` : Prend une collection de `Task` et retourne une `Task` qui se termine dès que **la PREMIÈRE tâche de la collection est terminée** (que ce soit par succès, échec ou annulation). C'est utile pour les scénarios de "premier arrivé, premier servi" ou de timeout.

Asynchrone avancé

L'Annulation des Tâches (`CancellationTokenSource`)

Pour éviter le gaspillage de ressources, il est essentiel de fournir un mécanisme pour **annuler** les opérations asynchrones longues (par exemple, si l'utilisateur ferme l'application ou si une requête met trop de temps).

- **`CancellationTokenSource`** : L'objet qui gère le jeton (token).
- **`CancellationToken`** : Le jeton passé aux méthodes asynchrones (comme `HttpClient.GetAsync(url, token)`). La méthode vérifie périodiquement l'état de ce jeton.
- **Mécanisme** : Lorsque `CancellationTokenSource.Cancel()` est appelé, les tâches qui surveillent le jeton lèvent une exception `OperationCanceledException`.

Explication du Mécanisme d'Annulation 🖐️

Le modèle d'annulation en C# utilise un système de jeton (token) coopératif :

1. **`CancellationTokenSource`** : C'est l'objet que le code appelant (**l'émetteur**) crée et contrôle. Il a une méthode `Cancel()` pour signaler l'arrêt.
2. **`CancellationToken`** : C'est le jeton lui-même, extrait du `Source`. Il est passé à la méthode asynchrone (**le destinataire**).
3. **`Destinataire`** : La méthode asynchrone doit périodiquement vérifier l'état du jeton (`token.IsCancellationRequested`) ou utiliser des méthodes prévues pour l'annulation (`token.ThrowIfCancellationRequested()`).

Lorsqu'une méthode reçoit le signal d'annulation, elle lève une exception **`OperationCanceledException`**, permettant au code appelant de gérer l'arrêt en douceur.